Check for updates

ASME Journal of Mechanical Design
Online journal at:
https://asmedigitalcollection.asme.org/mechanicaldesign



Gabriel Garayalde
Department of Civil and Environmental Engineering,
Politecnico di Milano,
Piazza L. Da Vinci 32,
Milano 20133, Italy
e-mail: gabrielemilio.garayalde@polimi.it

Luca Rosafalco
Department of Civil and Environmental Engineering,
Politecnico di Milano,
Piazza L. Da Vinci 32,
Milano 20133, Italy
e-mail: luca.rosafalco@polimi.it

Matteo Torzoni[1]
Department of Civil and Environmental Engineering,
Politecnico di Milano,
Piazza L. Da Vinci 32,
Milano 20133, Italy
e-mail: matteo.torzoni@polimi.it

Alberto Corigliano
Department of Civil and Environmental Engineering,
Politecnico di Milano,
Piazza L. Da Vinci 32,
Milano 20133, Italy
e-mail: alberto.corigliano@polimi.it

Mastering Truss Structure Optimization With Tree Search

This study investigates the combined use of generative grammar rules and Monte Carlo tree search (MCTS) for optimizing truss structures. Our approach accommodates intermediate construction stages characteristic of progressive construction settings. We demonstrate the significant robustness and computational efficiency of our approach compared to alternative reinforcement learning frameworks from previous research activities, such as Q-learning or deep Q-learning. These advantages stem from the ability of MCTS to strategically navigate large state spaces, leveraging the upper confidence bounds for trees formula to effectively balance exploitation–exploration trade-offs. We also emphasize the importance of early decision nodes in the search tree, reflecting design choices crucial for highly performative solutions. Additionally, we show how MCTS dynamically adapts to complex and extensive state spaces without significantly affecting solution quality. While the focus of this article is on truss optimization, our findings suggest that MCTS is a powerful tool for addressing other increasingly complex engineering applications.
[DOI: 10.1115/1.4068300]

Keywords: Monte Carlo tree search, truss optimization, reinforcement learning, computational design synthesis, agent-based design, computer-aided engineering, design optimization, design process, machine learning

1 Introduction

Machine learning (ML) is impacting engineering applications, from structural health monitoring [1,2] and predictive maintenance [3] to optimal flow control [4] and automation in construction [5]. Thanks to algorithmic advances and increased computational capabilities, there is a promise in enabling new approaches in computational design synthesis (CDS)—a multidisciplinary research field aimed at automating the generation of design solutions for complex engineering problems [6–8]. By integrating constraints related to the fabrication process, for instance, through physics-based simulation, CDS could unlock the potential of additive manufacturing in various fields [9], e.g., 3D concrete printing [10].

The effectiveness of traditional approaches to truss optimization, such as the ground-structure method [11,12], has been established through decades of research. However, these methods suffer from high computational complexity and solution instability [13,14]. Alternative strategies for discrete truss optimization rely on heuristic techniques, including genetic algorithms [15–17], particle swarm [18,19], differential evolution [20], and simulated annealing [21]. Nevertheless, the applicability of these methods is similarly limited by their high computational burden and slow convergence as the size of the search space increases [14].

The search space of candidate solutions can be narrowed through generative design grammars [22], which facilitate the exploration of alternative designs within a coherent framework [23,24]. These grammars are structured sets of rules that constrain the space of design configurations by accounting for mechanical information, such as the stability of static equilibrium. By integrating these rules within optimization procedures, it is therefore possible to explore incremental construction processes where the final design is reached through intermediate feasible configurations. The use of grammar-based approaches for truss topology generation and optimization has been proposed in Ref. [25], while their integration within heuristic approaches has been explored in Refs. [26–30].

Recently, the optimal truss design problem has been formalized as a Markov decision process (MDP) [31]. The solution to an MDP involves a series of choices, or actions, aimed at maximizing the long-term accumulation of rewards, which in this context measures the design objective. Viewing truss optimal design through the MDP lens, an action consists of adding or removing truss members, with the ultimate goal of optimizing a design objective, e.g., minimize the structural compliance. The final design thus emerges from a series of actions, possibly guided by grammar rules. This procedure is particularly suitable for truss structures, as it naturally

[1]Corresponding author.

Contributed by the Design Automation Committee of ASME for publication in the Journal of Mechanical Design. Manuscript received May 30, 2024; final manuscript received February 1, 2025; published online April 16, 2025. Assoc. Editor: Shikui Chen.

Copyright © 2025 by ASME; reuse license CC-BY 4.0

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

accommodates discrete structural optimization, where adding a single member can significantly alter the functional objective of the design problem. Additionally, it can be extended to design optimization in additive manufacturing settings and continuum mechanics. The same methodology is similarly applicable to parametric optimization problems [32], including cases with stochastic control variables [33].

Reinforcement learning (RL) is the branch of ML that addresses MDPs through repeated and iterative evaluations of how a single action affects a certain objective [34]. Relevant instances of RL-based optimization in engineering include two-dimensional kinematic mechanisms [35] and the ground structure of binary trusses [36]. The advantage of RL over heuristic methods lies in its flexibility in handling high-dimensional problems, as demonstrated in Refs. [37–39]. In Ref. [31], the MDP formalizing the optimal truss design has been solved using Q-learning [40], constraining the search space with the grammar rules proposed in Ref. [41]. In a separate work [42], the same authors have also addressed the challenges of large and continuous design spaces through *deep* Q-learning.

In this article, we demonstrate how addressing optimal truss design problems with the Monte Carlo tree search (MCTS) algorithm [43,44] can offer significant computational savings compared to both Q-learning and deep Q-learning. MCTS is the RL algorithm behind the successes achieved by "AlphaGo" [45] and its successors [46,47] in playing board games and video games. In science and engineering, MCTS has been used for various applications employing its single-player games version [48]. Notable instances include protein folding [49], materials design [50,51], fluid-structure topology optimization [52], and the optimization of the dynamic characteristics of reinforced concrete structures [53].

For truss design, MCTS has been used in "AlphaTruss" [54] to achieve state-of-the-art performance while adhering to constraints on stress, displacement, and buckling levels. The same framework has been extended to handle continuous state-action spaces through either kernel regression [55] or soft actor-critic [56]—an off-policy RL algorithm. Despite the potential of using continuous descriptions of the design problem, the combination of RL and grammar rules proposed in Refs. [31,42] remains highly competitive, as it enables constraining the design process with strong inductive biases reflecting engineering knowledge. Building on this insight, the novelty of our approach lies in the integration of MCTS with grammar rules to strategically navigate the solution space, allowing for significant computational gains compared to Refs. [31,42], where Q-learning and deep Q-learning have been respectively adopted.

The effectiveness of the proposed approach lies in the MCTS capability to propagate information from the terminal nodes of the tree, which are associated with the final design performance, back to the ancestor nodes linked with the initial design states. This feedback mechanism allows for informing subsequent simulations, exploiting previously synthesized designs to enhance the decision-making process at initial branches and progressively refine the search toward optimal designs. Moreover, the probabilistic nature of MCTS enables the discovery of highly performative design solutions by balancing the exploitation–exploration trade-off. This balance is achieved through a heuristic hyperparameter that tunes the upper confidence bounds for trees (UCT) formula, whose effect is investigated through a parametric analysis.

The remainder of the article is organized as follows. Section 2 states the optimization problem and provides an overview of the MDP setting, grammar rules, and the MCTS algorithm. In Sec. 3, the computational procedure is assessed on a series of case studies. We provide comparative results with respect to Refs. [31,42], demonstrating superior design capabilities, and we test our methodology on two novel progressive construction setups. Section 4 finally summarizes the obtained results and draws the conclusions.

2 Methodology

In this section, we describe the methodology characterizing our optimal truss design strategy. This includes the physics-based numerical model behind the design problem in Sec. 2.1, the MDP formalizing the design process in Sec. 2.2, the grammar rules for truss design synthesis in Sec. 2.3, the MCTS algorithm for the optimal truss design formulated as an MDP in Sec. 2.4, and the UCT formula behind the selection policy in Sec. 2.5, before detailing their algorithmic integration in Sec. 2.6.

2.1 Optimal Truss Design Problem. The design problem involves defining the truss geometry that optimizes a design objective under statically applied loading conditions. In the following, we consider minimizing the maximum absolute displacement experienced by the structure, although this is not a restrictive choice. This design setting, similar to the compliance minimization problem typical of topology optimization [13], has been retained for the purpose of comparison with Refs. [31,42].

For the sake of generality, we set the design problem in the context of a continuum elasticity, of which truss design is an immediate specialization. Specifically, we seek a set of I subdomains $\{\Omega_1^s,\dots,\Omega_I^s\}$, each occupying a certain region of the design domain, whose union $\Omega=\bigcup_{i=1}^{I}\Omega_i^s$ minimizes the structure's displacement to the greatest extent, as follows:

$$\bar{\Omega}=\bigcup_{i=1}^{I}\bar{\Omega}_i^s=\underset{\Omega=\bigcup_{i=1}^{I}\Omega_i^s}{\arg\min}\ \|\mathbf{u}(\mathbf{x})\|_\infty,\quad \text{with } \mathbf{x}\in\Omega \tag{1}$$

where $\mathbf{u}$ is the displacement field, $\mathbf{x}$ are the spatial coordinates, and $\|\mathbf{u}\|_\infty$ is the infinity norm of $\mathbf{u}$, defined as $\|\mathbf{a}\|_\infty=max_m|a_m|$, with a_m, $m=1,\dots,M$, being the mth entry of $\mathbf{a}\in\mathbb{R}^M$. Problem (1) is subjected to the following constraints, $\forall\Omega=\bigcup_{i=1}^{I}\Omega_i^s$:

$$\nabla\cdot\boldsymbol{\sigma}+\mathbf{b}=0\quad\text{equilibrium} \tag{2a}$$

$$\boldsymbol{\sigma}=\mathbf{E}\boldsymbol{\epsilon}\quad\text{linear elastic constitutive law} \tag{2b}$$

$$\boldsymbol{\epsilon}=\frac{\nabla\mathbf{u}+\nabla\mathbf{u}^\top}{2}\quad\text{linear kinematic compatibility} \tag{2c}$$

ensuring the static elasticity condition. Herein, $\boldsymbol{\sigma}$ is the stress field, $\boldsymbol{\epsilon}$ is the strain field, $\mathbf{b}$ is the vector of body forces, $\mathbf{E}$ is the elasticity tensor, $(\nabla\cdot)$ is the divergence operator, and (∇) is the gradient operator.

Moreover, problem (2) needs to be equipped with the following set of boundary conditions (BCs):

$$\mathbf{u}=\mathbf{u}_g,\quad\text{on }\partial\Omega_g=\bigcup_{i=1}^{I}\partial\Omega_{g_i}^s\quad\text{Dirichlet BC} \tag{3a}$$

$$\boldsymbol{\sigma}\cdot\mathbf{n}=\mathbf{f},\quad\text{on }\partial\Omega_h=\bigcup_{i=1}^{I}\partial\Omega_{h_i}^s\quad\text{Neumann BC} \tag{3b}$$

where $\partial\Omega_g$ and $\partial\Omega_h$ are the Dirichlet and Neumann boundaries, respectively; $\mathbf{u}_g$ is the assigned displacement field on $\partial\Omega_g$; $\mathbf{f}$ is the vector of surface tractions acting on $\partial\Omega_h$; and $\mathbf{n}$ is the outward unit vector normal to $\partial\Omega_h$. It is worth highlighting that this framework can be generalized to include nonlinear constitutive behaviors—an extension that will be explored in future works.

Equation (1) can be easily adapted for planar trusses by introducing a finite element (FE) discretization to solve problem (2), defining each subdomain Ω_i^s, $i=1,\dots,I$, to be a truss element, with the union set operator representing the connections made through hinges. Accordingly, the optimization problem is reformulated as

$$\min_{\Omega=\bigcup_{i=1}^{I}\Omega_i^s}\ \|\mathbf{U}(\Omega)\|_\infty,\quad\text{with }\Omega_i^s\text{ a truss FE} \tag{4a}$$

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

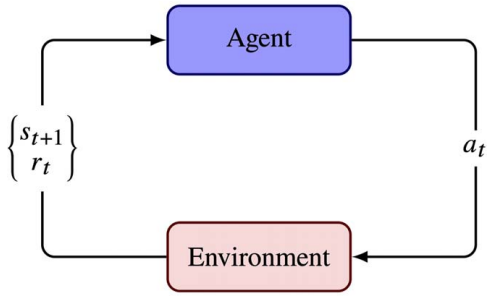


Fig. 1 Schematic agent–environment interaction

$$\text{subject to:} \quad \mathbf{KU} = \mathbf{F}, \quad \text{in } \Omega = \bigcup_{i=1}^{I} \Omega_i \tag{4b}$$

$$\mathbf{U} = \mathbf{U}_0, \quad \text{on } \partial\Omega_g = \bigcup_{i=1}^{I} \partial\Omega_{g_i}^{s} \tag{4c}$$

$$V \leq V^{\max}, \quad \text{with } V = \sum_{i=1}^{I} A_i L_i \tag{4d}$$

where $\mathbf{U}$ is the vector of nodal displacements; $\mathbf{K}$ is the stiffness matrix; $\mathbf{F}$ is the vector of forces induced by the external loadings; $\mathbf{U}_0$ is the vector of nodal displacements enforced on $\partial\Omega_g$; A_i and L_i are the cross-sectional area and the length of the ith truss element Ω_i^s, respectively; and $V^{\max}$ is a prescribed threshold on the maximum allowed volume of the truss lattice. For further details on the FE method, the reader may refer to Ref. [57].

2.2 Markov Decision Process Framework for Sequential Decision Problems. In a decision-making setting, an *agent* must choose from a set of possible actions, each potentially leading to uncertain effects on the state of the system. The decision-making process aims to maximize, at least on average, the numerical utilities assigned to each possible action outcome. This involves considering both the probabilities of various outcomes and our preferences among them.

In sequential decision problems, the agent's utility is influenced by a sequence of decisions. MDPs provide a framework for describing these problems in fully observable, stochastic *environments* with Markov transition models and additive rewards [58]. Formally, an MDP is a four-tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$, comprising a space of states $\mathcal{S}$ that the system can assume, a space of actions $\mathcal{A}$ that can be taken, a Markov transition model $\mathcal{P}$, and a space of rewards $\mathcal{R}$. The characterization of these quantities for truss optimization purposes is detailed below, after discussing their roles in MDPs.

We consider a time discretization of a planning horizon $(0, T)$ using nondimensional time-steps $t = 0, \ldots, T$, and we denote the system state at time t as $s_t \in \mathcal{S}$, which is the realization of the random variable $S_t \sim p(s_t)$, with $p(s_t)$ being the probability distribution encoding the relative likelihood that $S_t = s_t$. Moreover, we denote the control input at time t as $a_t \in \mathcal{A}$. The transition model $\mathcal{P}: \mathcal{S} \times \mathcal{S} \times \mathcal{A} \mapsto [0, 1]$ encodes the probability of reaching any state s_{t+1} at time $t + 1$, given the current state s_t and an action a_t, i.e., $p(s_{t+1} \mid s_t, a_t) \in \mathcal{P}$. The reward $R_t \sim p(r_t)$, with $r_t \in \mathcal{R}$, quantifies the value associated with each possible set $\{s_t, a_t, s_{t+1}\}$.

We define a control policy $\pi: \mathcal{S} \mapsto \mathcal{A}$ as the mapping from any system state to the space of actions. The goal is to find the *optimal* control policy $\pi^*(S_t)$ that provides the optimal action a_t^* for each possible state s_t. The optimal policy $\pi^*(S_t)$ is learned by identifying the action a_t^* that maximizes the expected utility over $(0, T)$. The problem of finding the optimal control policy is inherently stochastic. Consequently, the associated objective function is additive and relies on expectations [59]. This is typically expressed as the total expected discounted reward over $(0, T)$.

The sequential decision problem can be viewed from the perspective of an agent–environment interaction, as depicted in Fig. 1. In this view, the agent perceives the environment and aims to maximize the long-term accumulation of rewards by choosing an action a_t that influences the environment at time $t + 1$. The environment interacts with the agent by defining the evolution of the system state, and providing a reward r_t for taking a_t and moving to s_{t+1}.

One way to characterize an MDP is to consider the expected utility associated with a policy $\pi(S_t)$ when starting in any state s_t and following $\pi(S_t)$ thereafter. To this aim, the *state-value* function $V^{\pi}(S_t): \mathcal{S} \mapsto \mathbb{R}$ quantifies, for every state s_t, the total expected reward an agent can accumulate starting in s_t and following policy $\pi(S_t)$. In contrast, the *action-value* function, $Q^{\pi}(S_t, a_t): \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$, reflects the expected accumulated reward starting from s_t, taking action a_t, and then following policy $\pi(S_t)$. In both cases, the probability of reaching any state s_{t+1} is estimated using transition probabilities $p(s_{t+1} \mid s_t, a_t)$.

For our purposes of optimal truss design, we refer to a grid-world environment with a predefined number of nodes on which possible truss layouts can be defined. The reward function shaping $\mathcal{R}$ could account for local design objectives, such as the displacement at a prescribed node, or global performance indicators, such as the maximum absolute displacement, stress level, or strain energy. As previously commented, we monitor the maximum absolute displacement experienced by the structure. The state space $\mathcal{S}$ could potentially include any feasible truss layout resulting from progressive construction processes. Accordingly, the space of actions $\mathcal{A}$ could account for any possible modification of a given layout. In this scenario, the sizes of $\mathcal{S}$ and $\mathcal{A}$ increase significantly, even considering a reasonably small design domain. For this reason, explicitly modeling the Markov transition model $\mathcal{P}$ is not feasible.

The availability of a transition model for an MDP influences the selection of appropriate solution algorithms. Dynamic programming algorithms, for instance, require explicit transition probabilities. In situations where representing the transition model becomes challenging, a simulator is often employed to implicitly model the MDP dynamics. This is typical in episodic RL, where an environment simulator is queried with control actions to sample environment trajectories of the underlying transition model. Examples of such algorithms include Q-learning, as seen in Refs. [31,42], and MCTS, both of which approximate the action-value function and use this estimate as a proxy for the optimal control policy. As noted in Ref. [44], convergence to the global optimal value function can only be guaranteed asymptotically in these cases. In our truss design problem, optimal planning is achieved via simulated experience provided by the FE model in Eq. (4*b*), which can be queried to produce a sample transition given a state and an action.

2.3 Grammar Rules for Truss Design Synthesis. To introduce the grammar rules that we employ to guide the process of optimal design synthesis, we refer to a starting seed configuration s_0, defined by deploying a few bars to create a statically determinate truss structure. This initial configuration must be modified through a series of actions selected by an agent. Every time an allowed action is enacted on the current state s_t, a new configuration s_{t+1} is generated (see Fig. 2). The process continues until reaching a state s_T, characterized by a terminal condition, such as achieving the maximum allowed volume $V^{\max}$ of the truss members.

To identify the allowed actions, we use the same grammar rules as those used in Refs. [31,41,42]. Starting from an isostatic seed configuration, these rules constrain the space of design configurations by allowing only truss elements resulting in triangular forms to be added to the current configuration, thereby ensuring statically determinate configurations. Given any current configuration s_t, an allowed action is characterized by a sequence of three operations:

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

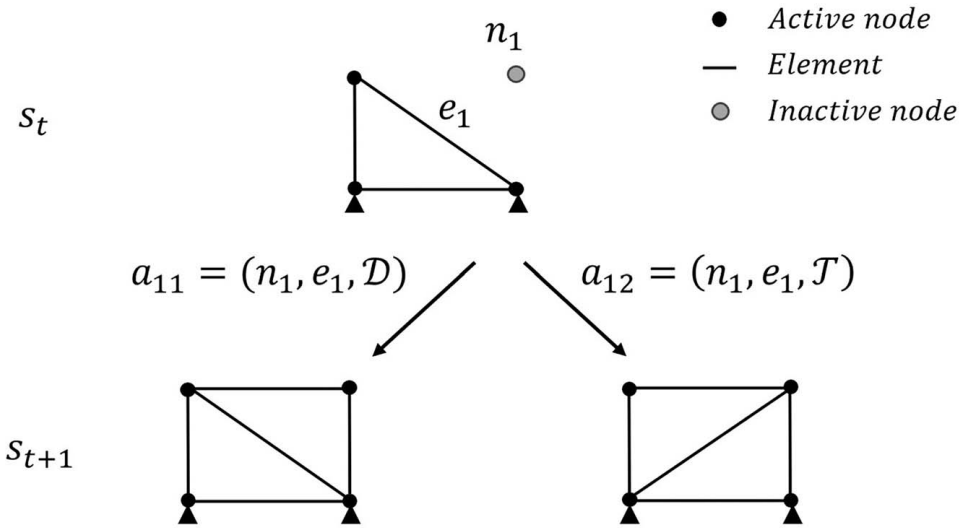


Fig. 2 Exemplary actions following operators $\mathcal{D}$ and $\mathcal{T}$. The current configuration s_t (top) is modified either through action a_1 (bottom left) following the $\mathcal{D}$ operator or through action a_2 (bottom right) following the $\mathcal{T}$ operator, resulting in a new configuration s_{t+1}. In both cases, the selected truss element is e_1, and the chosen inactive node is n_1.

(1) Choosing a node among those not yet reached by the already placed truss elements. We term these nodes as *inactive*, to distinguish them from the previously selected *active* nodes.
(2) Selecting a truss element already in place.
(3) Applying a legal operator based on the position of the chosen node with respect to the selected element. The legal operators are either "$\mathcal{D}$" or "$\mathcal{T}$," see also Fig. 2. A $\mathcal{D}$ operator adds the new node and links it to the current configuration without removals, while a $\mathcal{T}$ operator also removes the selected element before connecting the new node. In both cases, the connections to the new node are generated ensuring no intersection with existing elements.

2.4 Monte Carlo Tree Search. The MCTS algorithm is a decision-time planning RL method [34]. It relies on two fundamental principles: (i) approximating action-values through random sampling of simulated environment trajectories and (ii) using these estimates to inform the exploration of the search space, progressively refining the search toward highly rewarding trajectories.

In the context of optimal truss design formulated as a sequential decision-making problem, MCTS incrementally grows a search tree where each node represents a specific design configuration and edges correspond to potential state transitions triggered by allowed actions (see Fig. 3). During training, the algorithm explores the search space of feasible truss designs to progressively learn a control policy, referred to as the *tree policy*. This progressive policy improvement is based on value estimates of state-action pairs derived from previous runs of the algorithm, termed episodes. Each episode consists of four main phases [34]:

(1) *Selection*: Starting from the root node associated with the seed configuration, the algorithm traverses the tree by selecting child nodes according to the tree policy until reaching a leaf node. The tree policy typically uses the UCT formula [44] to select child nodes. This formula ensures that actions leading to promising nodes are more likely to be chosen while still allowing for the exploration of less-visited nodes.
(2) *Expansion*: If the selected leaf node corresponds to a nonterminal state s_t, the algorithm expands the tree by adding one or more child nodes representing unexplored actions from s_t. This expansion phase introduces new potential design configurations into the search tree, broadening the scope of exploration.
(3) *Simulation (rollout)*: From one of the newly added nodes, the algorithm performs a path simulation or "rollout" to estimate the value gained by passing from that node. Since the tree policy does not yet cover the newly added nodes, MCTS employs a *rollout policy* during this simulation phase to pick actions until reaching a terminal state s_T. The rollout policy is a random policy satisfying the truss design grammar rules, directing action along unexplored paths to backpropagate the associated reward signal back up the decision tree. While the tree policy expands the tree via selection and expansion, the rollout policy simulates environment interaction based on random exploration.
(4) *Backpropagation*: Upon reaching a terminal state s_T, the associated design is synthesized to evaluate the design objective. This reward is then backpropagated through the nodes traversed during selection and expansion. This process involves updating the visit counts of the nodes and the values $Q^\pi(s, a)$ for the corresponding state-action pairs, both of which influence the decision-making process through the UCT formula, as detailed in the following section.

Each time a reward signal is backpropagated to update the action-value estimates of state-action pairs, an episode is completed. This iterative process progressively refines the tree policy, making actions that lead to better rewards more likely to be chosen in future episodes, while still allowing for the exploration of new design configurations. The number of episodes is determined by the available computational budget. As the number of episodes increases, more nodes are added to the tree, and the precision of the Monte Carlo estimates for the mean return from each state-action pair improves. After completing the prescribed number of episodes, a deterministic policy can be derived by selecting, for example, the action with the highest estimated value $Q^\pi(s, a)$ at each state.

The advantages of MCTS stem from its online, incremental, sample-based value estimation and policy improvement. MCTS is particularly adept at managing environments where rewards are not immediate, as it effectively explores broad search spaces despite the minimal feedback. This makes MCTS especially suitable for progressive construction settings, where the final design

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

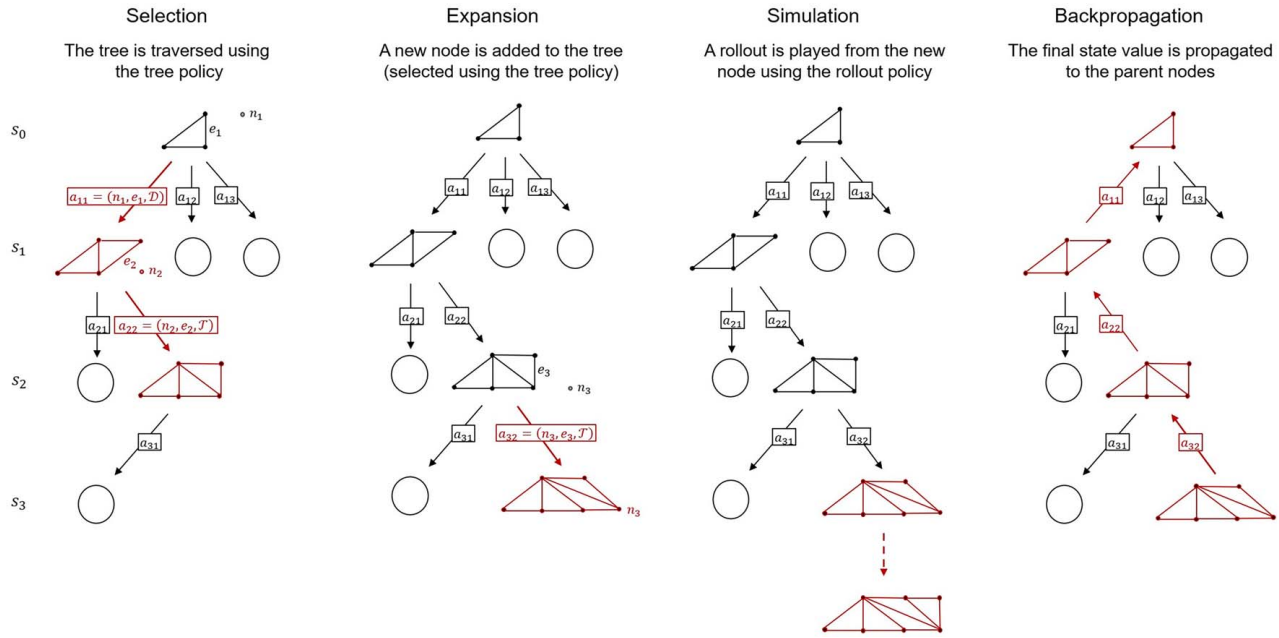


Fig. 3 Exemplary use of grammar rules for the optimal truss design, formalized as a Markov decision process and solved through Monte Carlo tree search. The search tree construction and the corresponding truss design synthesis are achieved by repeating the four steps of selection, expansion, simulation, and backpropagation.

requirements often differ from those of intermediate structural states. Intermediate construction stages typically involve sustaining self-load only, while different combinations of dead and live loads are experienced during operations. This capability stems from the backpropagation step, which allows information related to s_T to be transferred to the early nodes of the tree. In contrast, bootstrapping methods like Q-learning may require a longer training phase to equivalently backpropagate information, as we demonstrate in Sec. 3. Further advantages of MCTS include: (i) accumulating experience by sampling environment trajectories, without requiring domain-specific knowledge to be effective; (ii) incrementally growing a lookup table to store a partial action-value function for the state-action pairs yielding highly rewarding trajectories, without needing to approximate a global action-value function; (iii) updating the search tree in real-time whenever the outcome of a simulation becomes available, in contrast, e.g., with minimax's iterative deepening; and (iv) focusing on promising paths thanks to the selective process, leading to an asymmetric tree that prioritizes more valuable decisions. This last aspect not only enhances the algorithm's efficiency but can also offer insights into the domain itself by analyzing the tree's structure for patterns of successful courses of action.

2.5 Upper Confidence Bounds for Trees. The UCT formula is widely used as a selection policy in MCTS due to its ability to balance exploitation and exploration. In this work, we employ a modified UCT formula, compared to the one proposed in Ref. [44], by introducing an α parameter that scales the relative weights of the exploitation and exploration terms as follows:

$$\mathrm{UCT}_j = (1-\alpha)\frac{v_j^{\Sigma}}{n_j} + \alpha\sqrt{\frac{2\log\sum_l n_l}{n_j}} \tag{5}$$

which provides the UCT score of the jth child node of s_t. Herein, v_j^{Σ} is the Monte Carlo estimate of the total return gained by passing through the jth child node, where this return represents the sum of all the terminal state rewards r_T achieved after traversing the jth child node; n_j is the number of episode runs passing through the jth child node; $\sum_l n_l$ is the total number of episode runs traversing the children of s_t; and α is a parameter that balances exploitation (average reward for the jth child node) and exploration (encouraging exploration of nodes that have been visited less frequently than their siblings), respectively encoded in the first and second terms. It is also worth noting that both v_j^{Σ} and n_j are updated after each training episode.

2.6 Algorithmic Description. The algorithmic description of the optimal truss design strategy using the proposed MCTS approach is detailed in Algorithm 1. It begins by initializing the root node with a seed configuration and then iteratively explores potential truss configurations through a sequence of selection, expansion, simulation, and backpropagation phases. In each episode, the algorithm selects a child node based on the UCT formula, generates and evaluates a new child node from a possible action, simulates random descendant nodes to explore the design

Table 1 Truss optimization—problem setting description

	Domain size	Decision times, T	$V^{\max}$ threshold
Case 1	4×3	2	160
Case 2	5×3	3	240
Case 3	5×5	3	225
Case 4	5×9	3	305
Case 5	5×5	4	480
Case 6	7×7	4	350

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

space, and backpropagates the computed reward to update the policy.

Algorithm 1 Monte Carlo tree search for optimal truss design

input: number of episodes N_e
parametrization of the physics-based model
grid design domain
seed configuration
grammar rules for truss design synthesis
exploration parameter α
initialize root node for the seed configuration
for N_e **do**
  $t = 0$
  set root node for $s_{t=0}$ (seed configuration)
  ▷selection
  while $t < T$ and s_t previously explored **do**
    select s_{t+1} via UCT formula
    $t \leftarrow t + 1$
  ▷expansion
  if $t < T$ and s_t not previously explored **then**
    for states s_{t+1} from allowed actions a_t **do**
      solve static equilibrium for s_{t+1}
      compute design objective $\| U(\Omega)\|_\infty$
    $t \leftarrow t + 1$
  ▷simulation
  while $t < T$ **do**
    select a random child s_{t+1}
    if s_{t+1} not previously explored **then**
      solve static equilibrium for s_{t+1}
      compute design objective $\| U(\Omega)\|_\infty$
    $t \leftarrow t + 1$
  ▷backpropogation
  compute reward r_T from terminal state s_T
  while $t > 0$ **do**
    append r_T to s_t rewards list
    s_t visit count += 1
    $t \leftarrow t - 1$
return deterministic control policy $\pi \approx \pi^*$

3 Results

In this section, we assess the proposed MCTS framework on different truss optimization problems. First, we adopt six case studies from Refs. [31,42], each featuring different domain and boundary conditions, to directly compare the achieved performance. Then, we consider two additional case studies to demonstrate the applicability of our procedure for progressive construction purposes. While in the former case studies the seed configuration fully covers the available design domain, in the latter we allow the seed configuration to grow—mimicking an additive construction process—until reaching a terminal node at the far end of the domain.

The experiments have been implemented in `PYTHON` using the `Spyder` development environment. All computations have been carried out on a PC featuring an `AMD Ryzen™ 9 5950X` CPU @ 3.4 GHz and 128 GB RAM.

3.1 Truss Optimization. In the following, we present the results achieved for the six case studies adapted from Refs. [31,42], providing comparative insights for each scenario. All case studies deal with planar trusses, with truss elements featuring dimensionless Young's modulus $E = 10^3$ and cross-sectional area $A = 1$. The applied forces have a dimensionless value of $f_x = f_y = 10$, as per Refs. [31,42]. The *monitored* displacement refers to the maximum absolute displacement experienced by the structure.

Each row of Table 1 describes a case study in terms of design domain size, number of decision times or planning horizon T, and volume threshold $V^{\max}$. These parameters have been set according to Refs. [31,42] to facilitate the comparison between the proposed MCTS procedure and the Q-learning methods. For each case study, the design domain, structural seed configuration, externally applied force(s), and boundary conditions are shown under the corresponding s_0 label in Fig. 4. The target *optimal* configuration s_T, identified through a brute-force exhaustive search of the state space, is illustrated under the s_T label. Case Study 4 is the only one that differs from the reference due to the additional constraint at (0, 0).

For each case study, Fig. 5 shows the evolution of the design objective, i.e., the maximum absolute displacement experienced by the structure, as the number of training episodes increases. Results are reported in terms of average displacement (solid line) and one-standard-deviation credibility interval (shaded area), over ten independent training runs. Each run utilizes MCTS for a predefined number of episodes. In practice, the number of episodes is set after an initial long training run in which we assess the number of episodes required to achieve convergence—which typically depends on the complexity of the case study. After each training run, the best configuration is saved to subsequently compute relevant statistics. The attained displacement values are compared with those associated with the global minima (dashed lines), representing the optimal design configurations in Fig. 4.

The heuristic α parameter in Eq. (5) controls the balance between exploitation and exploration. The α values employed for the six case studies are overlaid on each learning curve in Fig. 5. Since an optimal value for this parameter is not known a priori, this is set using a rule of thumb derived through a parametric analysis, as explained in the following section for case study 4.

A quantitative assessment of the optimization performance for each case study is summarized in Table 2. Results are reported in terms of the optimal design objective $\|\mathbf{U}(\overline{\Omega})\|_\infty$, the percentage ratio of the optimal design objective to the displacement achieved by the learned policy, and the percentile score relative to the exhaustive search space. To clarify, a percentile score of 100% corresponds to reaching the global optimum. A lower score, such as 99%, indicates that the design objective achieved with the final design s_T, synthesized from the learned optimal policy, is lower than the displacement associated with 99% of all the possible configurations explored through an exhaustive search. An exemplary distribution of the design objective across the population of designs synthesized from the exhaustive search of the state space is shown in Fig. 6 for case study 4. Interestingly, the distributions obtained for the other case studies also exhibit a lognormal-like shape, although these are not shown here due to space constraints. While the objective ratio provides a dimensionless measure of how close the achieved design is to the global optimum in terms of performance, the percentile score quantifies the capability of MCTS to navigate the search space and find a design solution close to the optimal one. Both performance indicators are computed by averaging over ten training runs. Additionally, we report the number of FE evaluations required to achieve a near-optimal or optimal policy, also averaged over ten training runs, and indicate the percentage savings in the number of FE evaluations compared to those required by the deep Q-learning strategy from Ref. [42]. It is worth noting that FE evaluations are only performed for the terminal state s_T, after it has been selected.

3.2 Case Study 4—Detailed Analysis. In this section, we provide a detailed analysis of case study 4. We selected this case study because it is the only one in which we employ boundary conditions different from those reported in Ref. [42], which is useful for checking the MCTS capability to exploit constrained domain portions not included in the seed configuration. Figure 7 illustrates the sequence of structural configurations synthesized from the optimal policy obtained through an exhaustive search. For each

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

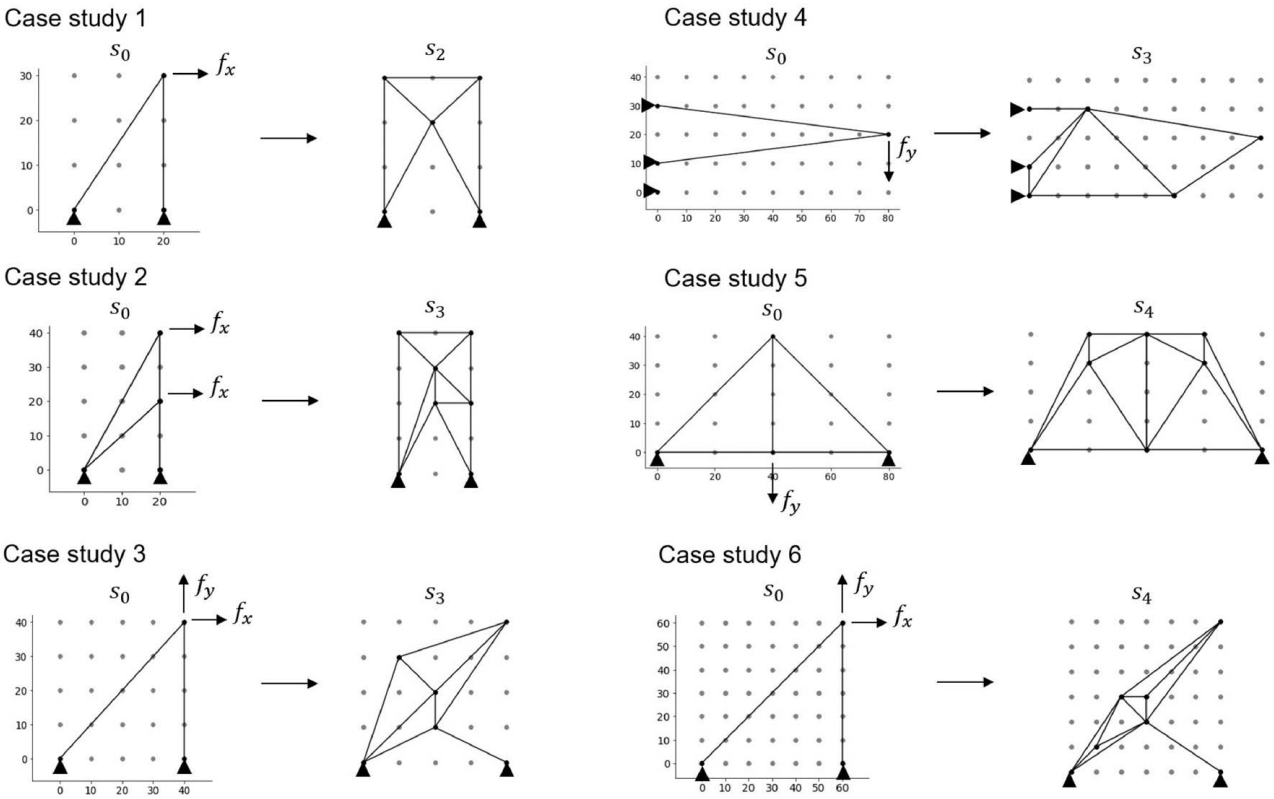


Fig. 4 Truss optimization—case studies adapted from Ref. [42]: summary of design domain, seed configuration s_0, and target optimal design s_T identified through a brute-force exhaustive search

decision time, we report the corresponding value of the design objective and the volume of the truss lattice below the synthesized configuration s_t.

Figure 8 summarizes the impact of varying the α parameter on the attained percentile score to provide insights into the selection of an appropriate value. Specifically, Fig. 8(*a*) shows the percentile score relative to the exhaustive search space for different α values, averaged over ten training runs. Figure 8(*b*) illustrates how the percentile score evolves as the number of episodes increases, offering insights into the effect of α on the convergence of MCTS. To compare the achieved performance for varying α values with the associated computational burden, Fig. 9 presents the number of FE evaluations required to achieve a near-optimal design policy, revealing an almost linear increase in the number of FE evaluations as α grows. Therefore, we consider $\alpha = 0.3$ to provide an appropriate balance between exploitation and exploration, yielding an average percentile score of 99.90% across ten training runs, which is close to the scores for $\alpha = 0.4$ and $\alpha = 0.5$, but with only

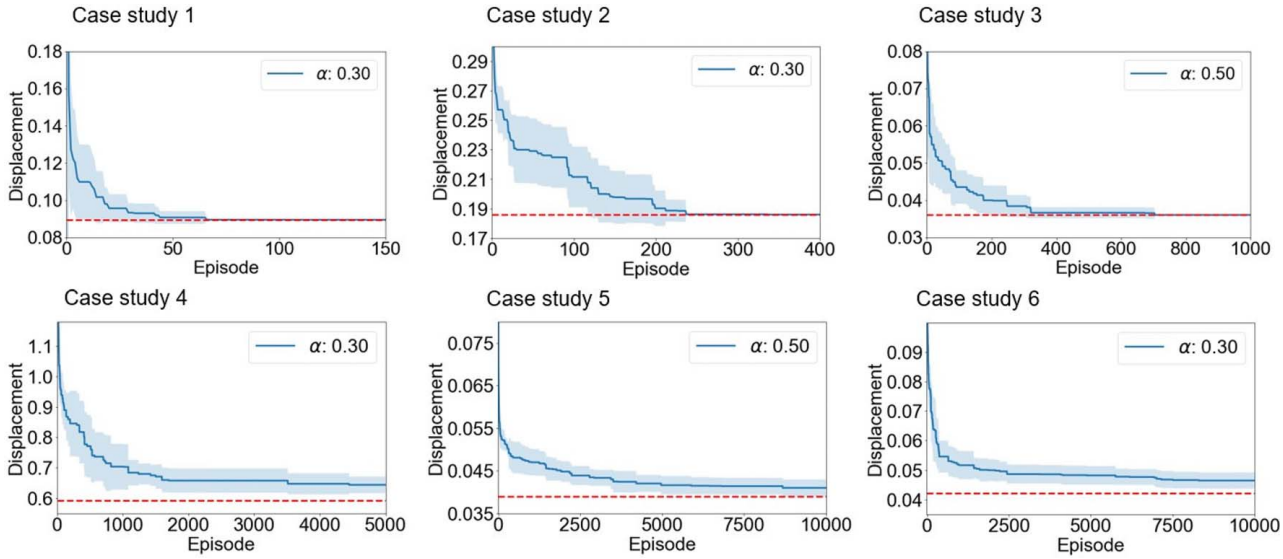


Fig. 5 Truss optimization—case studies 1–6: evolution of the design objective during training, shown as the average value (solid line) with its one-standard-deviation credibility interval (shaded area) and target global minimum (dashed line). Results averaged over ten training runs.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

Table 2 Truss optimization—case studies 1–6: optimal design objective $\|\mathbf{U}(\overline{\Omega})\|_\infty$, percentage ratio of the optimal design objective to the displacement achieved by the learned policy, percentile score relative to the exhaustive search space, number of finite element evaluations required to achieve a near-optimal or optimal policy, and relative speed-up compared to Ref. [42]. The speed-up is not reported for case study 4, as it differs from the reference for the additional constraint at (0, 0). Results averaged over ten training runs.

	$\|\mathbf{U}(\overline{\Omega})\|_\infty$	Objective ratio (%)	Percentile score (%)	FE runs	FE runs versus Ref. [42] (%)
Case 1	0.0895	100	100	106	−74.70
Case 2	0.1895	100	100	517	−76.27
Case 3	0.0361	100	100	966	−56.51
Case 4	0.5916	91.91	99.90	1672	N/A
Case 5	0.0390	95.23	99.99	9739	−70.74
Case 6	0.0420	90.44	99.98	7931	−31.25

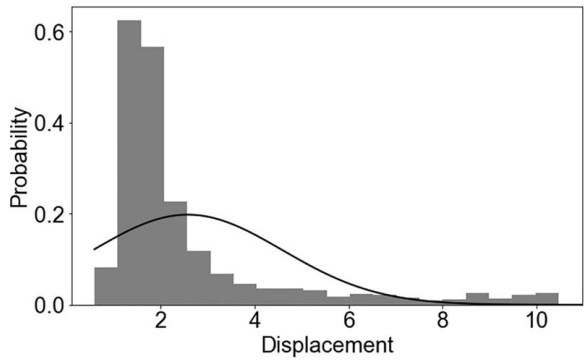


Fig. 6 Truss optimization—case study 4: design objective distribution over the population of designs synthesized from an exhaustive search of the state space

1672 FE evaluations. The achieved ratio of the optimal design objective to the displacement achieved by the learned policy is 91.91% (see Table 2). Similar results from the parametric analysis of α for the other case studies are provided in Appendix A.

3.3 Progressive Construction. In this section, we showcase the potential of the proposed MCTS strategy in guiding the progressive construction of a truss cantilever beam and a bridge-like structure. Unlike in the previous case studies, where a simplified seed configuration was initially assigned to comply with the target boundary conditions and then refined, here we allow the seed configuration to progressively grow until reaching a prescribed terminal node not included in the initial configuration. Therefore, the agent must account for the intermediate construction stages per se, not just as necessary steps to reach the final configuration. Another difference compared to the previous case studies is that instead of considering a fixed loading configuration, the structure

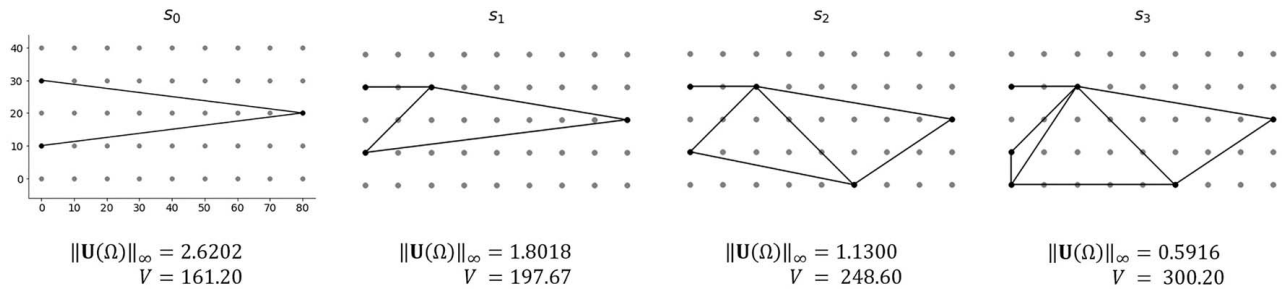


Fig. 7 Truss optimization—case study 4: sequence of design configurations from the target optimal policy, identified through a brute-force exhaustive search, with details about the design objective value and the truss lattice volume

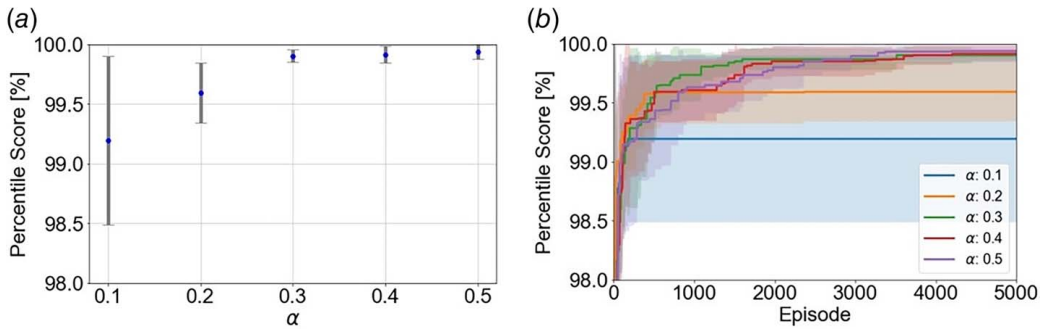


Fig. 8 Truss optimization—case study 4: impact of varying the α parameter on the attained percentile score relative to the exhaustive search space. For each value of α, results are reported in terms of (*a*) the average percentile score with its one-standard-deviation credibility interval and (*b*) the evolution of the percentile score during training, shown as the average value with its credibility interval. Results averaged over ten training runs.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

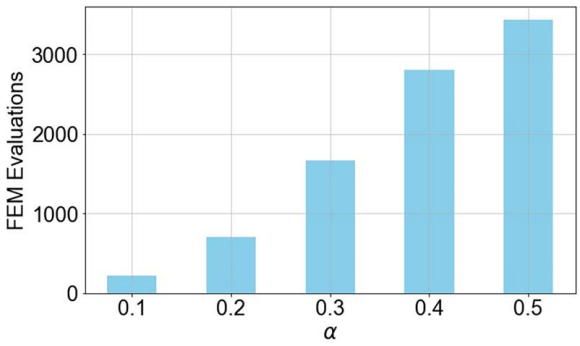


Fig. 9 Truss optimization—case study 4: number of finite element evaluations required to achieve a near-optimal design policy for varying values of α parameter

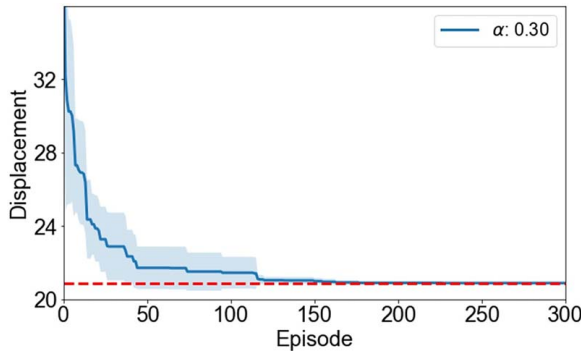


Fig. 11 Progressive construction—cantilever case study: evolution of the design objective during training, shown as the average value (solid line) with its one-standard-deviation credibility interval (shaded area), and target global minimum (dashed line). Results averaged over ten training runs.

is subjected to self-weight (unit dimensionless density), modifying the loading configuration at each stage. However, as in the previous cases, since the design process aims to maximize the performance of the final configuration, the chosen design objective is again the maximum absolute displacement. Although we did not set a limit on the maximum number of states, the agent must strike a balance between achieving higher structural stiffness by adding additional members and the weight these extra elements bring.

For the cantilever case study, we assign a domain size of 50×20, while for the bridge-like case study, we consider a larger domain of 80×30, which features a central passive area where FEs cannot be connected. The sequence of optimal design configurations is shown in Fig. 10 for the cantilever beam and in Fig. 12 for the bridge-like structure. These optimal sequences have been synthesized from an exhaustive search, halted due to computational constraints after scanning 10,000,000 and 1,200,000 possible configurations, respectively. The maximum length of the individual elements has been constrained to comply with the typical fabrication, transportation, and on-site assembly limitations encountered in construction projects. This realistic constraint compels the algorithm to explore more detailed designs, avoiding trivial configurations that rely on only a few long elements to reach the target node.

For the cantilever case study, the optimal configuration is synthesized 100% of the time over ten training runs. Using $\alpha = 0.3$, MCTS requires an average of 507 FE evaluations per training run, yielding an optimal displacement of 20.861. It is worth noting how the algorithm identifies the optimal configuration by focusing on the most promising solutions, which feature more elements near the clamped side rather than near the free end (see s_7 in Fig. 10). Refer to Fig. 11 for the evolution of the attained design objective as the number of episodes increases during training.

Similarly, for the bridge-like case study, the MCTS policy synthesizes the optimal configuration 100% of the time over ten training runs. The evolution of states from the MCTS policy is identical to that of the target optimal policy, as shown in Fig. 12. Using $\alpha = 0.3$, the algorithm requires an average of 901 FE evaluations per training run, yielding an optimal displacement of 7.147. Finally, Fig. 13 presents the corresponding evolution of the attained design objective during training.

These case studies highlight the advantages of MCTS over Q-learning approaches for optimal design synthesis in large state spaces. In such cases, Q-learning struggles because it requires sufficient sampling of each state-action pair to build a Q-table that stores values for every possible pair, leading to exponential growth in memory and computational demands as the number of states increases. In contrast, MCTS dynamically builds a decision tree based on the most promising moves explored through simulation, focusing computational resources on more relevant parts of the search space. This selective exploration allows MCTS to handle large state spaces more efficiently than Q-learning, making it better suited to problems where direct enumeration of all state-action pairs is infeasible. Appendix B provides an overview of the computational burden associated with a pilot MCTS training of 1000 episodes for the bridge-like case study.

3.4 Discussion. In the case studies used for comparison with Ref. [42], the proposed MCTS framework has been capable of synthesizing a near-optimal solution with significantly fewer FE evaluations. Case study 2 has shown the greatest reduction, requiring

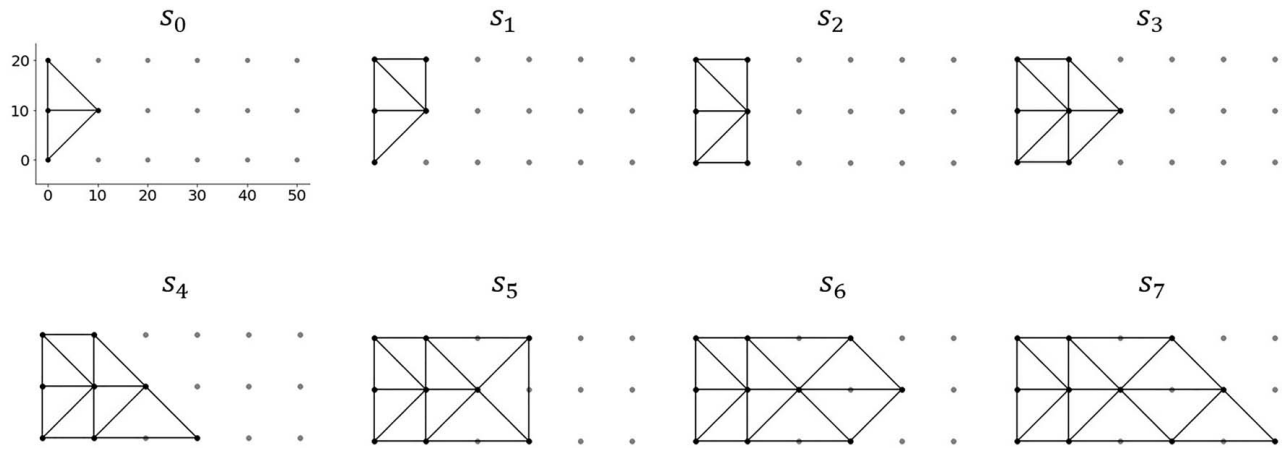


Fig. 10 Progressive construction—cantilever case study: sequence of design configurations from the target optimal policy

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

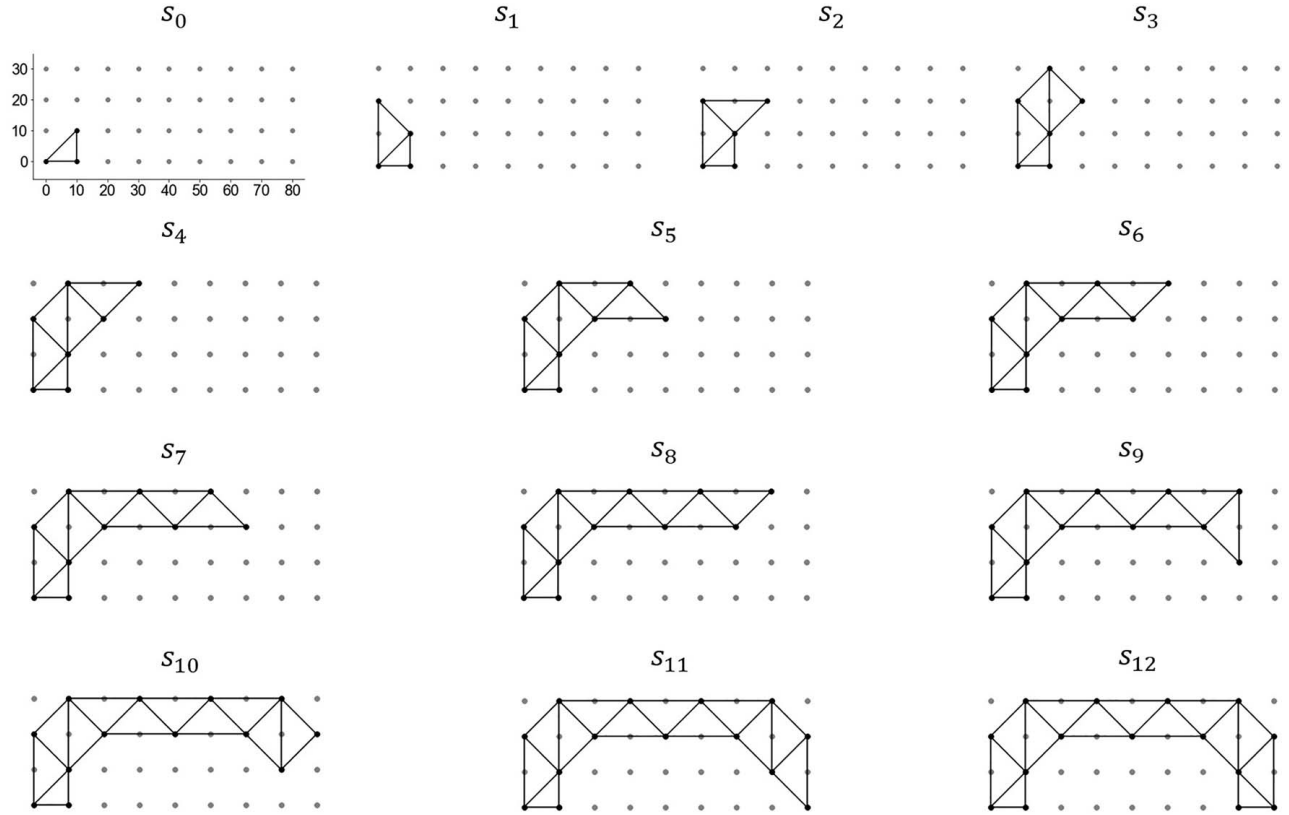


Fig. 12 Progressive construction—bridge-like case study: sequence of design configurations from the target optimal policy

76% fewer evaluations. All examples have achieved a percentile score greater than 99.9% when compared to the candidate solutions from the exhaustive search. In the first three case studies, the global optimal solution has been synthesized 100% of the time. However, this was not the case for the last three.

As seen in Table 2, the lowest percentile score has been achieved in case study 4. A significant factor contributing to this result is the large number of decision nodes present in the first layer of the search tree. Although case study 4 features a smaller search space compared to case study 5, it exhibits a considerably larger branching factor at the initial layer, with 94 child nodes as opposed to 42 in case study 5. This increased number of initial options introduces a greater degree of complexity, slowing down the algorithm's convergence rate. Wider trees are more computationally intensive, as they potentially feature promising paths hidden among a multitude of branches.

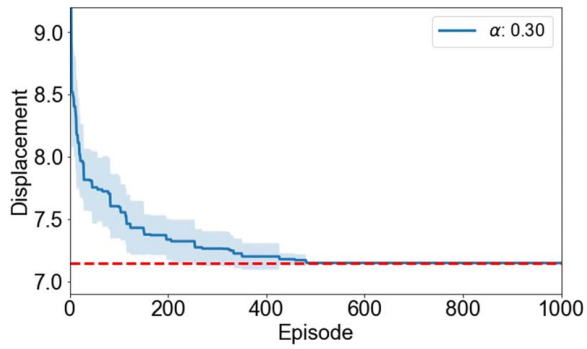


Fig. 13 Progressive construction—bridge-like case study: evolution of the design objective during training, shown as the average value (solid line) with its one-standard-deviation credibility interval (shaded area), and target global minimum (dashed line). Results averaged over ten training runs.

This aspect may lead to insufficient exploration of the branches departing from each node, resulting in less accurate value estimation. A straightforward solution to improve the approximation accuracy is to increase the number of episodes, thereby allowing each child node more visits. Another factor contributing to the lower percentile score in case study 4 is that the optimal solution is located in a section of the tree crowded with numerous lower-quality configurations. The UCT formula (5) favors the exploitation of areas of the tree that, on average, yield good results. Consequently, the procedure tends to overlook tree branches that could potentially lead to the global optimum in favor of branches that consistently yield good configurations. The rationale is that areas of the tree yielding, on average, better solutions typically also contain the optimal configuration. However, this is not necessarily true for every problem, and it is not the case for case study 4. Even though reaching the global optimum is not precluded, this limitation of the UCT formula may slow convergence to the optimum. To address these drawbacks, the UCT equation could be modified to account for the standard deviation and the maximum value of the reward obtained from traversing the branch of child nodes, similarly to Refs. [48,60], as follows:

$$\mathrm{UCT}_j^{\mathrm{new}} = (1-\alpha)\left[\frac{(1-\beta)v_j^{\Sigma}}{n_j} + \beta v_j^{\mathrm{best}}\right] + \alpha\sqrt{\frac{2\log\sum_l n_l}{n_j} + \mathrm{var}(v_j)} \tag{6}$$

where v_j^{best} and $\mathrm{var}(v_j)$ represent the maximum value and the variance of the reward gained by passing through the jth child node, respectively, while β is a hyperparameter that balances exploitation between tree areas consistently yielding good results and areas containing the best-seen state. This modified selection policy is less likely to be unfairly biased toward nodes with fewer children and promotes increased exploration.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

The heuristic α parameter balances the exploitation and exploration terms in the UCT formula. From Fig. 8(*b*), we observe that for both $\alpha = 0.1$ and $\alpha = 0.2$, MCTS converges early on local minima. This occurs because the first term in the UCT formula dominates the second term, preventing sufficient exploration of child nodes in the tree. While increasing the value of α mitigates this issue, the number of FE evaluations is observed to rise linearly with α, as shown in Fig. 9. However, the greater computational burden required by $\alpha = 0.4$ and $\alpha = 0.5$ results only in a limited improvement in the percentile score compared to $\alpha = 0.3$, as shown in Fig. 8. Thus, $\alpha = 0.3$ has been chosen to balance a high percentile score with a low number of FE evaluations.

The cantilever and bridge-like case studies discussed in Sec. 3.3, within the context of progressive construction, demonstrate the potential of the proposed MCTS framework to synthesize optimal designs in problems with very large state spaces. The (partial) exhaustive search has analyzed more than 10, 000, 000 possible design configurations for the cantilever beam and 1, 200, 000 for the bridge-like structure. Despite this, the optimal solution has been synthesized 100% of the time, requiring only 507 FE evaluations per training run for the cantilever beam and 901 for the bridge-like structure. In contrast, case study 4, which features a much smaller state space, has required significantly more FE evaluations without achieving the global optimum. This discrepancy is partly due to the fact that, although there are more layers in the trees of the progressive construction case studies, each layer is much narrower, allowing the algorithm to more easily identify promising branches. For instance, in the cantilever case, the first layer of the decision tree has 15 times fewer nodes compared to case study 4, resulting in much lower complexity.

One strength of MCTS over Q-learning [31] and deep Q-learning [42] approaches to optimal design synthesis is its ability to backpropagate reward signals to ancestor nodes in the tree more effectively. The difficulty deep Q-learning faces in performing the backpropagation step has also been mentioned in Ref. [42]. Another limitation of Q-learning is that the reward signal is not continuously positive. Q-learning updates Q-values based on the difference between future and current reward estimates, adjusting only the values for the state-action pairs experienced in each step. These incremental updates cannot take place before backpropagating information from the final stage to the intermediate design configurations. These drawbacks are not present in the MCTS framework because the reward is computed at the end of every episode, as typical in Monte Carlo approaches, in contrast to temporal difference methods like Q-learning. Another key advantage of MCTS is that it builds a tree incrementally and selectively, exploring parts of the state space that are more promising based on previous episodes. This selective expansion is particularly advantageous in environments with extremely large or infinite state spaces, where attempting to maintain a value for every state-action pair (as in Q-learning) becomes infeasible. While we may not synthesize the absolute global optimum solution every time, we are able to achieve very high percentile scores. Most importantly, this framework can scale to large state spaces without significantly compromising the relative quality of the solutions.

4 Conclusion

This study has presented a comprehensive analysis of combining MCTS and generative grammar rules to optimize the design of planar truss lattices. The proposed framework has been tested across various case studies, demonstrating its capability to efficiently synthesize near-optimal (if not optimal) configurations even in large state spaces, with minimal computational burden. Specifically, we have compared MCTS with a recently proposed approach based on deep Q-learning, achieving significant reductions in the number of required finite element evaluations, ranging from 31% to 76% across different case studies. Moreover, two novel case studies have been used to highlight the adaptability of MCTS to dynamic and large state spaces typical of progressive construction scenarios. A critical analysis has been carried out to explain why the method has been able to get close to the global optimum without reaching it in some cases. Specifically, we have noted that this difficulty is not solely connected to the size of the state space but is due to the width of the tree and the adopted UCT formula. This formula encourages the exploration of tree sections with the best average reward, potentially neglecting sections with lower average rewards that may contain the global optimum.

Compared to Q-learning, the proposed MCTS-based strategy has demonstrated two key advantages: (i) an improved capability of backpropagating reward signals and (ii) the ability to selectively expand the decision tree toward more promising paths, thereby addressing large state spaces more efficiently and effectively.

The obtained results underscore the potential of MCTS not only in achieving high-percentile solutions but also in its scalability to large state spaces without compromising solution quality. As such, this framework is poised to be a robust tool in the field of structural optimization and beyond, where complex decision-making and extensive state explorations are required. In the future, modifications to the UCT formula will be explored to address the occasional challenges in reaching the global optimum. Moreover, we foresee the possibility of exploiting this approach in progressive construction, extending beyond the domain of planar truss lattices.

Acknowledgment

The authors of this article would like to thank Eng. Syed Yusuf and Professor Matteo Bruggi (Politecnico di Milano) for the invaluable insights and contributions during our discussions.

Funding Data

- This work is partly supported by ERC advanced grant IMMENSE—101140720. (Funded by the European Union. Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.)
- Matteo Torzoni acknowledges the financial support from Politecnico di Milano through the interdisciplinary PhD grant "Physics-Informed Deep Learning for Structural Health Monitoring."

Conflict of Interest

There are no conflicts of interest.

Data Availability Statement

The datasets generated and supporting the findings of this article are obtainable from the corresponding author upon reasonable request.

Appendix A: Parametric Analysis of the α Value

In this appendix, we provide results from a parametric analysis of the exploitation–exploration parameter α for case studies 1–6. Our modified UCT formula (5) employs the α parameter to control the relative scaling of the exploitation and exploration terms, in contrast to the standard UCT formula shown below:

$$\mathrm{UCT}_j = \frac{v_j^{\Sigma}}{n_j} + c\sqrt{\frac{2\log\sum_l n_l}{n_j}} \tag{A1}$$

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

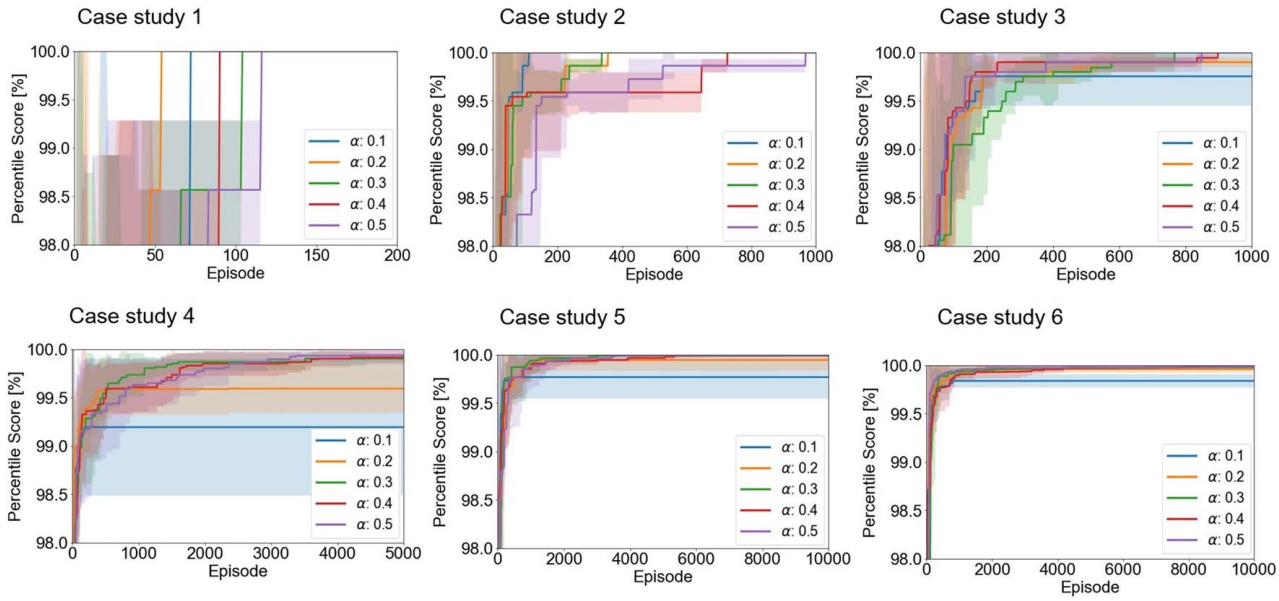


Fig. 14 Truss optimization—case studies 1–6: impact of varying the α parameter on the attained percentile score relative to the exhaustive search space. For each value of α, results are reported in terms of the evolution of the percentile score during training, shown as the average value with its one-standard-deviation credibility interval. Results averaged over ten training runs.

Table 3 Truss optimization—results for case studies 1–6: impact of varying the α parameter on the attained percentile score relative to the exhaustive search space, and on the number of finite element evaluations required to achieve a near-optimal or optimal policy. Results averaged over ten training runs.

		$\alpha = 0.1$	$\alpha = 0.2$	$\alpha = 0.3$	$\alpha = 0.4$	$\alpha = 0.5$
Case study 1	Percentile	100%	100%	100%	100%	100%
	FE runs	63	128	106	279	279
Case study 2	Percentile	100%	100%	100%	100%	100%
	FE runs	173	545	517	978	988
Case study 3	Percentile	99.75%	99.90%	100%	100%	100%
	FE runs	268	623	961	966	966
Case study 4	Percentile	99.19%	99.59%	99.90%	99.91%	99.94%
	FE runs	217	710	1672	2800	3433
Case study 5	Percentile	99.77%	99.95%	99.99%	99.99%	99.99%
	FE runs	586	2699	7235	9717	9739
Case study 6	Percentile	99.83%	99.96%	99.98%	99.98%	99.99%
	FE runs	770	3270	7931	9204	9357

For $\alpha = 0.5$, the modified UCT formula in Eq. (5) coincides with Eq. (A1) when $c = 1$. The choice of $c = 1$ is based on theoretical bounds that optimize the exploitation–exploration balance for the general case of the multi-armed bandit problem, where rewards are normalized between 0 and 1 [61]. While this serves as a good starting point, determining the appropriate α value is challenging to ascertain a priori, as it is case-dependent, as demonstrated in Fig. 14. A small choice of α can heavily prioritize exploitation, as shown in Fig. 14 for case studies 3–6 with $\alpha = 0.1$–0.2, which plateau well below the global optimum. This is also summarized in Table 3, where we provide the percentile scores and the number of required FE evaluations at varying α for the six case studies. More FE runs means more unique terminal states and more branches explored in the tree.

Appendix B: Computational Cost Analysis

In this appendix, we provide an overview of the computational burden associated with a pilot MCTS training of 1000 episodes for the bridge-like case study. The timing analysis is summarized in Table 4, reporting the computational time taken by each MCTS phase. The simulation phase dominates the computational time, accounting for 82.97% of the total execution time, followed by the expansion phase at 16.89%. In contrast, the selection and backpropagation phases require significantly less time, contributing 0.09% and 0.04%, respectively. The remaining operations, collectively termed "Other," take up a minimal 0.01% of the total execution time.

We have identified that the most computationally demanding task is not the FE analysis itself, but rather the frequent execution of relatively simple geometric checking functions that determine whether a new configuration violates geometric constraints. Specifically, a

Table 4 Progressive construction—bridge-like case study: timing breakdown of the MCTS phases during a pilot training session of 1000 episodes

MCTS phase	Time (s)	Percentage
Selection	0.91	0.09%
Expansion	163.08	16.89%
Simulation	801.00	82.97%
Backpropagation	0.38	0.039%
Other	0.01	0.01%
Total elapsed	965.3872	100%

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

function that checks whether a line passes over an active node has been called 14,559,543 times during execution. This function, invoked primarily during the child node population process, has been responsible for a cumulative execution time of 438.77 s, representing 45.45% of the total runtime. These calls occurred in both the expansion and simulation phases, significantly contributing to the overall computational load despite the function's simplicity. Overall, in the simulation phase, the algorithm must populate more layers of children nodes than in the expansion phase, therefore contributing to its higher computational cost.

References

[1] Rosafalco, L., Manzoni, A., Mariani, S., and Corigliano, A., 2022, *Combined Model Order Reduction Techniques and Artificial Neural Network for Data Assimilation and Damage Detection in Structures*, Springer International Publishing, Cham, Chapter 16, pp. 247–259.

[2] Rosafalco, L., Torzoni, M., Manzoni, A., Mariani, S., and Corigliano, A., 2022, *Self-adaptive Hybrid Model/Data-Driven Approach to SHM Based on Model Order Reduction and Deep Learning*, Springer International Publishing, Cham, Chapter 9, pp. 165–184.

[3] Torzoni, M., Tezzele, M., Mariani, S., Manzoni, A., and Willcox, K. E., 2024, "A Digital Twin Framework for Civil Engineering Structures," Comput. Methods Appl. Mech. Eng., **418**(Part B), p. 116584.

[4] Fan, D., Yang, L., Wang, Z., Triantafyllou, M. S., and Karniadakis, G. E., 2020, "Reinforcement Learning for Bluff Body Active Flow Control in Experiments and Simulations," Proc. Natl. Acad. Sci. USA, **117**(42), pp. 26091–26098.

[5] Elmaraghy, A., Montali, J., Restelli, M., Causone, F., and Ruttico, P., 2023, Towards an AI-Based Framework for Autonomous Design and Construction: Learning From Reinforcement Learning Success in RTS Games, CAAD Futures 2023, Communications in Computer and Information Science, Vol. 1819, Delft, July 5–7, Springer Nature, pp. 376–392.

[6] Antonsson, E. K., and Cagan, J., 2001, *Formal Engineering Design Synthesis*, Cambridge University Press, Cambridge.

[7] Chakrabarti, A., 2002, *Engineering Design Synthesis*, Springer Science & Business Media, New York.

[8] Campbell, M. I., and Shea, K., 2014, "Computational Design Synthesis," Artif. Intell. Eng. Des. Anal. Manuf., **28**(3), pp. 207–208.

[9] Wangler, T., Lloret, E., Reiter, L., Hack, N., Gramazio, F., Kohler, M., and Bernhard, M., 2016, "Digital Concrete: Opportunities and Challenges," RILEM Tech. Lett., **1**, pp. 67–75.

[10] Rizzieri, G., Ferrara, L., and Cremonesi, M., 2024, "Numerical Simulation of the Extrusion and Layer Deposition Processes in 3D Concrete Printing With the Particle Finite Element Method," Comput. Mech., **73**, pp. 277–295.

[11] Dorn, W. S., Gomory, R. E., and Greenberg, H. J., 1964, "Automatic Design of Optimal Structures," J. Mec., **3**(6), pp. 25–52.

[12] Rozvany, G., 1997, *Topology Optimization in Structural Mechanics*, Springer Verlag, Vienna.

[13] Garayalde, G., Torzoni, M., Bruggi, M., and Corigliano, A., 2024, "Real-Time Topology Optimization Via Learnable Mappings," Int. J. Numer. Methods Eng., **125**(15), p. e7502.

[14] Ohsaki, M., 2010, *Optimization of Finite Dimensional Structures*, CRC Press, Boca Raton, FL.

[15] Holland, J. H., 1992, *Adaptation in Natural and Artificial Systems: An Introductory Analysis With Applications to Biology, Control and Artificial Intelligence*, MIT Press, Cambridge, MA.

[16] Permyakov, V., Yurchenko, V., and Peleshko, I., 2006, "An Optimum Structural Computer-Aided Design Using Hybrid Genetic Algorithm," Proceeding of the 2006 International Conference on Progress in Steel, Composite and Aluminium Structures, Rzeszow, Poland, June 21–23, Taylor & Francis, Rzeszow, pp. 819–826.

[17] Hooshmand, A., and Campbell, M. I., 2016, "Truss Layout Design and Optimization Using a Generative Synthesis Approach," Comput. Struct., **163**, pp. 1–28.

[18] Kennedy, J., and Eberhart, R., 1995, "Particle Swarm Optimization," Proceedings of the 1995 International Conference on Neural Networks, Perth, Australia, Nov. 27–Dec. 1, Vol. 4, IEEE, pp. 1942–1948.

[19] Luh, G.-C., and Lin, C.-Y., 2011, "Optimal Design of Truss-Structures Using Particle Swarm Optimization," Comput. Struct., **89**(23), pp. 2221–2232.

[20] Ho-Huu, V., Nguyen-Thoi, T., Vo-Duy, T., and Nguyen-Trang, T., 2016, "An Adaptive Elitist Differential Evolution for Optimization of Truss Structures With Discrete Design Variables," Comput. Struct., **165**, pp. 59–75.

[21] Lamberti, L., 2008, "An Efficient Simulated Annealing Algorithm for Design Optimization of Truss Structures," Comput. Struct., **86**(19), pp. 1936–1953.

[22] Cagan, J., 2001, *Engineering Shape Grammars: Where We Have Been and Where We Are Going*, Cambridge University Press, New York, Chap. *3*, pp. 65–92.

[23] Mullins, S., and Rinderle, J. R., 1991, "Grammatical Approaches to Engineering Design. Part I: An Introduction and Commentary," Res. Eng. Des., **2**(3), pp. 121–135.

[24] Alber, R., and Rudolph, S., 2004, "On a Grammar-Based Design Language That Supports Automated Design Generation and Creativity," Knowledge Intensive Design Technology, J. C. Borg, P. J. Farrugia, and K. P. C. Camilleri, eds., July 23–25, 2002, Springer, St. Julians, MLTA, pp. 19–35.

[25] Reddy, G., and Cagan, J., 1995, "An Improved Shape Annealing Algorithm for Truss Topology Generation," ASME J. Mech. Des., **117**(2A), pp. 315–321.

[26] Shea, K., 1997, "Essays of Discrete Structures: Purposeful Design of Grammatical Structures by Directed Stochastic Search," Ph.D. thesis, Carnegie Mellon University, Pittsburgh, PA.

[27] Shea, K., and Cagan, J., 1997, "Innovative Dome Design: Applying Geodesic Patterns With Shape Annealing," Artif. Intell. Eng. Des. Anal. Manuf., **11**(5), pp. 379–394.

[28] Puentes, L., Cagan, J., and McComb, C., 2020, "Heuristic-Guided Solution Search Through a Two-Tiered Design Grammar," ASME J. Comput. Inf. Sci. Eng., **20**(1), p. 011008.

[29] Königseder, C., Shea, K., and Campbell, M. I., 2013, "Comparing a Graph-Grammar Approach to Genetic Algorithms for Computational Synthesis of PV Arrays," CIRP Design 2012, Bangalore, India, Mar 28–31, Springer London, pp. 105–114.

[30] Fenton, M., McNally, C., Byrne, J., Hemberg, E., McDermott, J., and O'Neill, M., 2015, "Discrete Planar Truss Optimization by Node Position Variation Using Grammatical Evolution," IEEE Trans. Evol. Comput., **20**(4), pp. 577–589.

[31] Ororbia, M. E., and Warn, G. P., 2021, "Design Synthesis Through a Markov Decision Process and Reinforcement Learning Framework," ASME J. Comput. Inf. Sci. Eng., **22**(2), p. 021002.

[32] Rosafalco, L., De Ponti, J. M., Iorio, L., Ardito, R., and Corigliano, A., 2023, "Optimised Graded Metamaterials for Mechanical Energy Confinement and Amplification Via Reinforcement Learning," Eur. J. Mech. A Solids, **99**, p. 104947.

[33] Rosafalco, L., De Ponti, J. M., Iorio, L., Craster, R. V., Ardito, R., and Corigliano, A., 2023, "Reinforcement Learning Optimisation for Graded Metamaterial Design Using a Physical-Based Constraint on the State Representation and Action Space," Sci. Rep., **13**(1), p. 21836.

[34] Sutton, R. S., and Barto, A. G., 2018, *Reinforcement Learning: An Introduction*, MIT Press, Cambridge, MA.

[35] Vermeer, K., Kuppens, R., and Herder, J., 2018, "Kinematic Synthesis Using Reinforcement Learning," 44th Design Automation Conference, Quebec City, QC, Aug. 6–29, p. V02AT03A009.

[36] Zhu, S., Ohsaki, M., Hayashi, K., and Guo, X., 2021, "Machine-Specified Ground Structures for Topology Optimization of Binary Trusses Using Graph Embedding Policy Network," Adv. Eng. Software, **159**, p. 103032.

[37] Mazyavkina, N., Sviridov, S., Ivanov, S., and Burnaev, E., 2021, "Reinforcement Learning for Combinatorial Optimization: A Survey," Comput. Oper. Res., **134**, p. 105400.

[38] Bello, I., Pham, H., Le, Q. V., Norouzi, M., and Bengio, S., 2016, "Neural Combinatorial Optimization With Reinforcement Learning," preprint arXiv:1611.09940.

[39] Jeon, W., and Kim, D., 2020, "Autonomous Molecule Generation Using Reinforcement Learning and Docking to Develop Potential Novel Inhibitors," Sci. Rep., **10**(1), p. 22104.

[40] Watkins, C. J., and Dayan, P., 1992, "Q-Learning," Mach. Learn., **8**(3), pp. 279–292.

[41] Lipson, H., 2008, "Evolutionary Synthesis of Kinematic Mechanisms," Artif. Intell. Eng. Des. Anal. Manuf., **22**(3), pp. 195–205.

[42] Ororbia, M. E., and Warn, G. P., 2023, "Design Synthesis of Structural Systems as a Markov Decision Process Solved With Deep Reinforcement Learning," ASME J. Mech. Des., **145**(6), p. 061701.

[43] Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., and Colton, S., 2012, "A Survey of Monte Carlo Tree Search Methods," IEEE Trans. Comput. Intell. AI Games, **4**(1), pp. 1–43.

[44] Kocsis, L., and Szepesvári, C., 2006, "Bandit Based Monte-Carlo Planning," Proceedings of the 2006 European Conference on Machine Learning, Berlin, Germany, Sept. 18–22, Springer, pp. 282–293.

[45] Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Schrittwieser, J., and Antonoglou, I., 2016, "Mastering the Game of Go With Deep Neural Networks and Tree Search," Nature, **529**(7587), pp. 484–489.

[46] Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., and Hubert, T., 2017, "Mastering the Game of Go Without Human Knowledge," Nature, **550**(7676), pp. 354–359.

[47] Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., and Guez, A., 2020, "Mastering Atari, Go, Chess and Shogi by Planning With a Learned Model," Nature, **588**(7839), pp. 604–609.

[48] Schadd, M. P., Winands, M. H., Van Den Herik, H. J., Chaslot, G. M. B., and Uiterwijk, J. W., 2008, "Single-Player Monte-Carlo Tree Search," Computers and Games, Sept 29–Oct 1, Beijing, Springer, Berlin, pp. 1–12.

[49] Yang, X., Yoshizoe, K., Taneda, A., and Tsuda, K., 2017, "RNA Inverse Folding Using Monte Carlo Tree Search," BMC Bioinf., **18**(1), p. 468.

[50] Dieb, T. M., Ju, S., Shiomi, J., and Tsuda, K., 2019, "Monte Carlo Tree Search for Materials Design and Discovery," MRS Commun., **9**(2), pp. 532–536.

[51] Dieb, T. M., Ju, S., Yoshizoe, K., Hou, Z., Shiomi, J., and Tsuda, K., 2017, "MDTS: Automatic Complex Materials Design Using Monte Carlo Tree Search," Sci. Technol. Adv. Mater., **18**(1), pp. 498–503.

[52] Gaymann, A., and Montomoli, F., 2019, "Deep Neural Network and Monte Carlo Tree Search Applied to Fluid-Structure Topology Optimization," Sci. Rep., **9**(1), p. 15916.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025

[53] Rossi, L., Winands, M. H., and Butenweg, C., 2022, "Monte Carlo Tree Search as an Intelligent Search Tool in Structural Design Problems," Eng. Comput., **38**(4), pp. 3219–3236.
[54] Luo, R., Wang, Y., Xiao, W., and Zhao, X., 2022, "Alphatruss: Monte Carlo Tree Search for Optimal Truss Layout Design," Buildings, **12**(5), p. 641.
[55] Luo, R., Wang, Y., Liu, Z., Xiao, W., and Zhao, X., 2022, "A Reinforcement Learning Method for Layout Design of Planar and Spatial Trusses Using Kernel Regression," Appl. Sci., **12**(16), p. 8227.
[56] Du, W., Zhao, J., Yu, C., Yao, X., Song, Z., Wu, S., Luo, R., Liu, Z., Zhao, X., and Wu, Y., 2023, "Automatic Truss Design With Reinforcement Learning," Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, Macao, China.
[57] Belytschko, T., Liu, W., and Moran, B., 2000, *Nonlinear Finite Elements for Continua and Structures*, John Wiley & Sons, Ltd, Chichester, UK.
[58] Bellman, R., 1957, "A Markovian Decision Process," J. Math. Mech., **6**(5), pp. 679–684.
[59] Papakonstantinou, K. G., and Shinozuka, M., 2014, "Planning Structural Inspection and Maintenance Policies Via Dynamic Programming and Markov Processes. Part I: Theory," Reliab. Eng. Syst. Saf., **130**, pp. 202–213.
[60] Jacobsen, E. J., Greve, R., and Togelius, J., 2014, "Monte Mario: Platforming With MCTS," Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation, Vancouver, BC, Canada, July 12–Oct 16, pp. 293–300.
[61] Auer, P., Cesa-Bianchi, N., and Fischer, P., 2002, "Finite-Time Analysis of the Multiarmed Bandit Problem," Mach. Learn., **47**(2), pp. 235–256.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/147/10/101702/7453649/md-24-1369.pdf by guest on 26 November 2025